

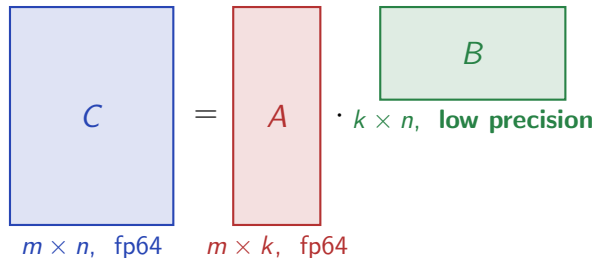
Cache-Optimal Tiling for Mixed-Precision Matrix Multiplication

Areg Mkhitarian Regev Ginzburg

Technion – Israel Institute of Technology

Supervisor: Alon Amid

Matrix Multiplication with Mixed Precision



Asymmetric element cost

	Precision	Bytes/elem
A	fp64	8
C	fp64	8
B	fp16	2
	or int8	1
	or PRNG FIFO	≈ 0

$$\rho = \frac{\text{bytes}(B)}{\text{bytes}(A)} < 1$$

Where Does Mixed Precision Arise?

Stored at lower precision

- Low-bit ML inference (fp16, bfloat16, int8)
- Quantized weight matrices
- Randomized sketching in linear algebra

Generated on the fly (PRNG)

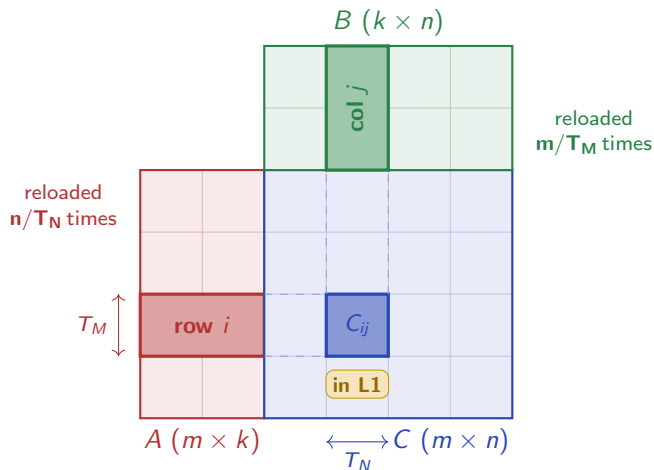
- B is a pseudorandom matrix
- Hardware FIFO streams random values at fixed rate

Question: Standard matmul (DGEMM) is optimized for $\rho = 1$.

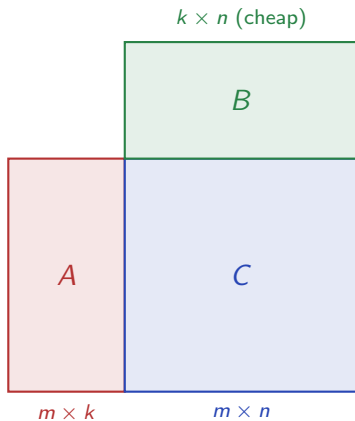
Can we do better by changing the **shape** of the computation tiles?

Tiling: The Standard Cache-Optimization Strategy

The fix (tiling): keep one tile of C resident in L1, stream A row-bands and B col-bands past it.

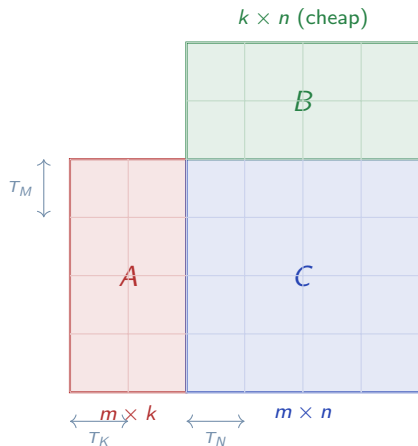


C-Stationary Tiling: Step by Step



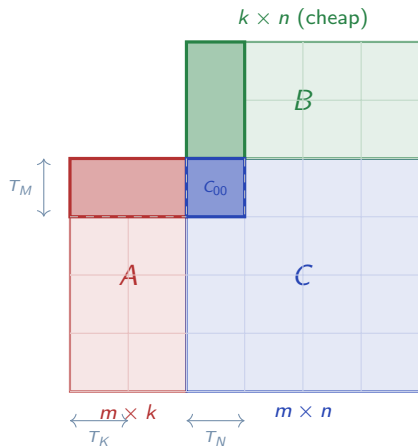
Three matrices. $C = A \cdot B$.

C-Stationary Tiling: Step by Step



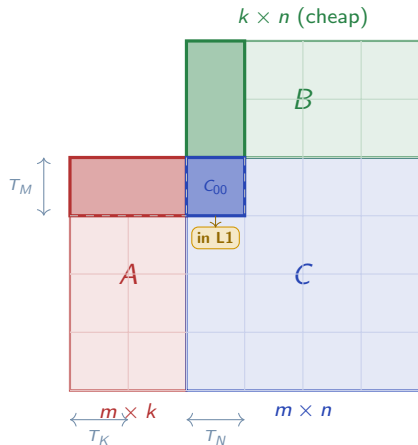
Partition into tiles of size $T_M \times T_N$ (T_K in the K-dimension).

C-Stationary Tiling: Step by Step



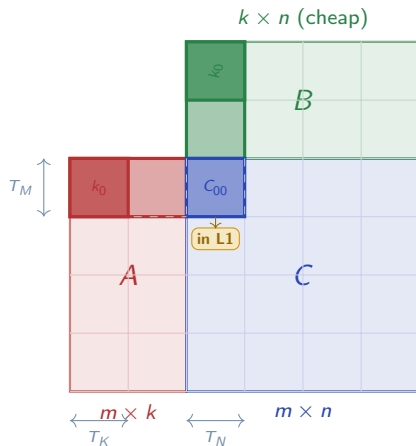
Select C_{00} — load corresponding A row-band and B col-band.

C-Stationary Tiling: Step by Step



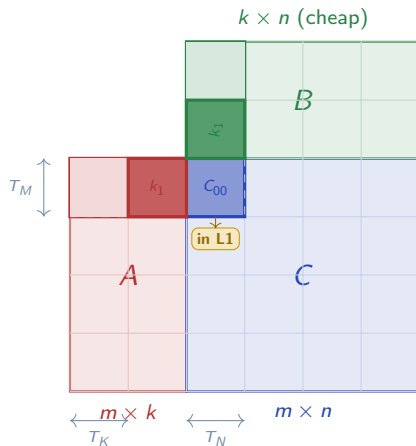
C_{00} enters L1 and **stays there** for all K iterations.

C-Stationary Tiling: Step by Step



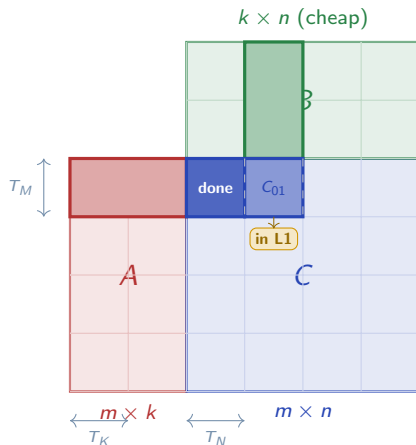
$$\text{K-loop } (k = 0): \quad C_{00} += A_{[i, k_0]} \times B_{[k_0, j]}$$

C-Stationary Tiling: Step by Step



K-loop ($k = 1$): $C_{00} += A_{[i, k_1]} \times B_{[k_1, j]}$ (only A/B K-chunks swap)

C-Stationary Tiling: Step by Step



C_{00} complete. Advance j : **swap B col only** — A row-band stays $\Rightarrow$ compute C_{01} .

The Memory Traffic Model

For one tile of C (size $T_M \times T_N$, fast memory $M = T_M \cdot T_N$):

Matrix	Reloads/elem	Cost
A (expensive)	n/T_N	$\propto \frac{1}{T_N}$
B (cheap, ρ)	m/T_M	$\propto \frac{\rho}{T_M}$

Minimize $\frac{1}{T_N} + \frac{\rho}{T_M}$ subject to $T_M \cdot T_N = M$.

Optimal tile shape

$$\frac{T_N}{T_M} = \frac{1}{\rho}$$

Wide tile when B is cheap ($\rho < 1$).

Examples:

- $\rho = 1$ (equal precision): $T_N = T_M$
(square)
- $\rho = 1/4$ (fp16 vs fp64):
 $T_N = 4 T_M$
- $\rho \rightarrow 0$ (PRNG FIFO): widest feasible tile

Deriving the Optimal Tile Shape

Parametrize by $z = T_M$ with constraint $T_M \cdot T_N = M$, so $T_N = M/z$:

$$f(z) = mnk \left(\frac{A_p \cdot z}{M} + \frac{B_p}{z} \right) + mn \cdot C_p$$

Differentiate and set to zero:

$$\begin{aligned} f'(z) &= mnk \left(\frac{A_p}{M} - \frac{B_p}{z^2} \right) = 0 \\ \Rightarrow z^2 &= \frac{B_p}{A_p} \cdot M = \rho M \end{aligned}$$

Substituting back ($z = T_M$, $M = T_M \cdot T_N$):

$$T_M^2 = \rho T_M T_N \Rightarrow T_M = \rho T_N \Rightarrow \boxed{\frac{T_N}{T_M} = \frac{1}{\rho}}$$

Analytical Speedup over Square Tiles

At the optimal tile the two traffic terms balance:

$$\underbrace{\frac{A_P}{T_N}}_{\text{A traffic}} = \underbrace{\frac{B_P}{T_M}}_{\text{B traffic}}$$

Substituting $T_N^* = \sqrt{M/\rho}$, $T_M^* = \sqrt{M\rho}$:

$$\text{speedup} = \frac{\text{cost}(\text{square})}{\text{cost}(\text{optimal})} = \frac{1 + \rho}{2\sqrt{\rho}}$$

Predicted speedups

ρ	B precision	speedup
1	fp64 (8 B)	1.00×
1/2	fp32 (4 B)	1.06×
1/4	fp16 (2 B)	1.25×
1/8	int8 (1 B)	1.59×
$\rightarrow 0$	PRNG FIFO	$\rightarrow \infty$

Speedup grows monotonically as B becomes cheaper.

Where the Speedup Formula Comes From

Traffic (ignoring C): $f = mnk \left(\frac{A_p}{T_N} + \frac{B_p}{T_M} \right)$

Square tile $T_M = T_N = \sqrt{M}$:

$$f_{\text{sq}} = \frac{mnk (A_p + B_p)}{\sqrt{M}}$$

Optimal tile $T_M^* = \sqrt{\rho M}$, $T_N^* = \sqrt{M/\rho}$:

$$f_{\text{opt}} = \frac{mnk}{\sqrt{M}} \left(A_p \sqrt{\rho} + \frac{B_p}{\sqrt{\rho}} \right)$$

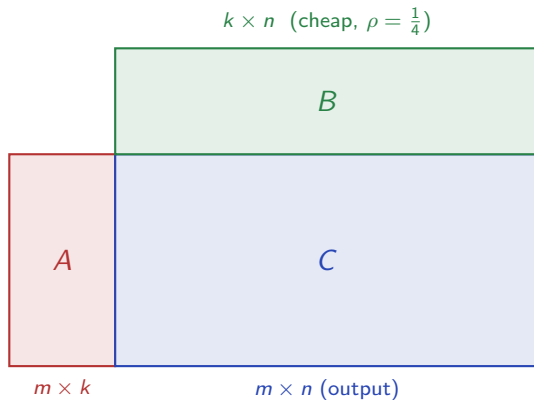
With $B_p = \rho A_p$: $A_p \sqrt{\rho} + \frac{\rho A_p}{\sqrt{\rho}} = 2A_p \sqrt{\rho}$

$$f_{\text{opt}} = \frac{2 mnk A_p \sqrt{\rho}}{\sqrt{M}}$$

$$\text{speedup} = \frac{f_{\text{sq}}}{f_{\text{opt}}} = \frac{A_p + B_p}{2A_p \sqrt{\rho}} = \frac{A_p(1 + \rho)}{2A_p \sqrt{\rho}} = \boxed{\frac{1 + \rho}{2\sqrt{\rho}}}$$

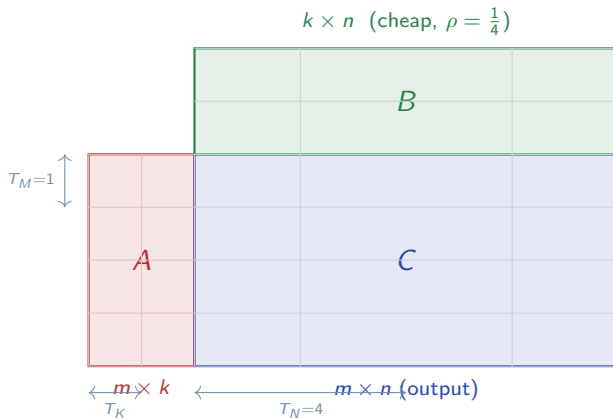
Note: at the optimum $A_p \sqrt{\rho} = B_p / \sqrt{\rho}$ — both input terms carry equal traffic.

C-Stationary Tiling with Asymmetric Precision ($\rho = \frac{1}{4}$)



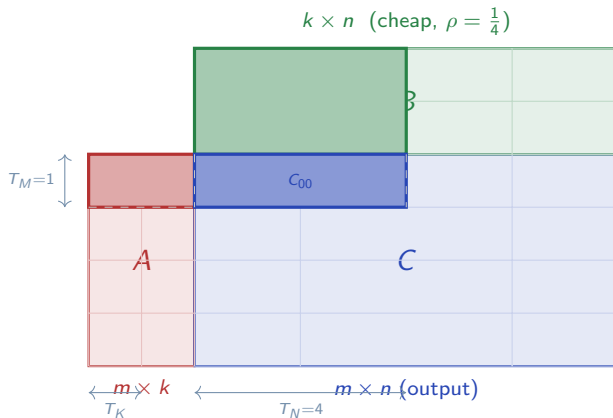
Wide matrices: C has more columns than rows.

C-Stationary Tiling with Asymmetric Precision ($\rho = \frac{1}{4}$)



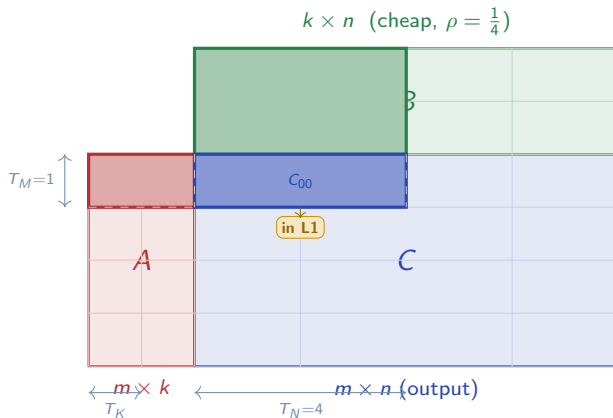
Wide tile: $T_N = 4 T_M$ — each C tile spans 4 columns, only 1 row.

C-Stationary Tiling with Asymmetric Precision ($\rho = \frac{1}{4}$)



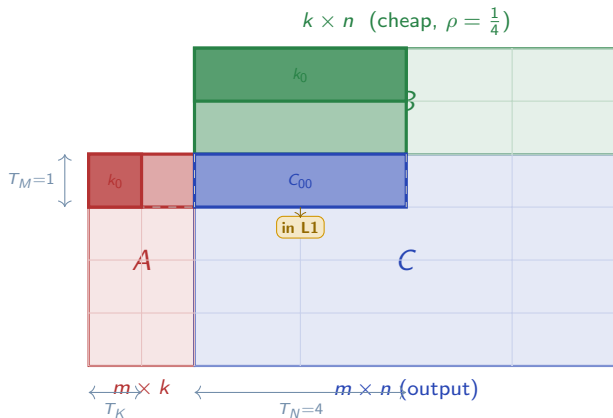
Select C_{00} (wide) — A row-band is **thin**; B col-band is **wide**.

C-Stationary Tiling with Asymmetric Precision ($\rho = \frac{1}{4}$)



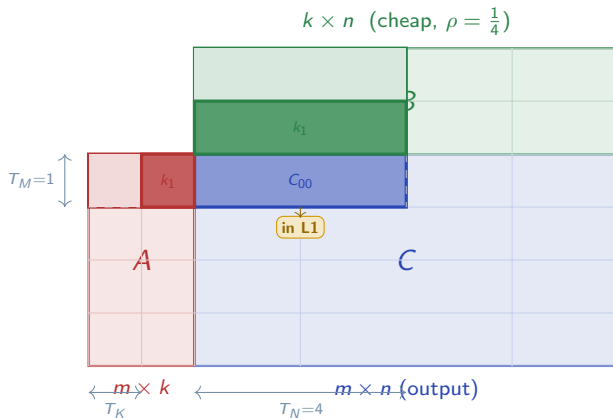
C_{00} enters L1 and stays. The wide C tile fits because T_M is small.

C-Stationary Tiling with Asymmetric Precision ($\rho = \frac{1}{4}$)



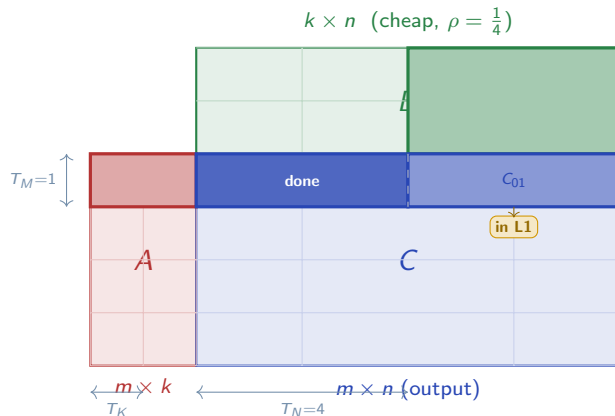
$$\text{K-loop } (k = 0): \quad C_{00} += A_{[i, k_0]} \times B_{[k_0, j:j+4]}$$

C-Stationary Tiling with Asymmetric Precision ($\rho = \frac{1}{4}$)



$$\text{K-loop } (k = 1): \quad C_{00} += A_{[i, k_1]} \times B_{[k_1, j:j+4]}$$

C-Stationary Tiling with Asymmetric Precision ($\rho = \frac{1}{4}$)



C_{00} done. Swap B col-band (2 total). A row stays. A reloads: $n/T_N = 2$. B reloads: $m/T_M = 4$ (cheap — OK!)

Cycle-Accurate Cache Simulator

How it works

- An **instruction stream generator** emits a .matv trace for a given tile shape and ordering
- A **cycle-accurate interpreter** replays the trace through L1 + L2 + DRAM
- Output: total cycles, per-level traffic, hit/miss counts

Instruction set

- `ltea` — load a register tile from memory
- `tmove` — consume values from the PRNG FIFO
- `prefetch` — issue a prefetch hint
- `tmulac` — register-tile multiply-accumulate

Two modes for B

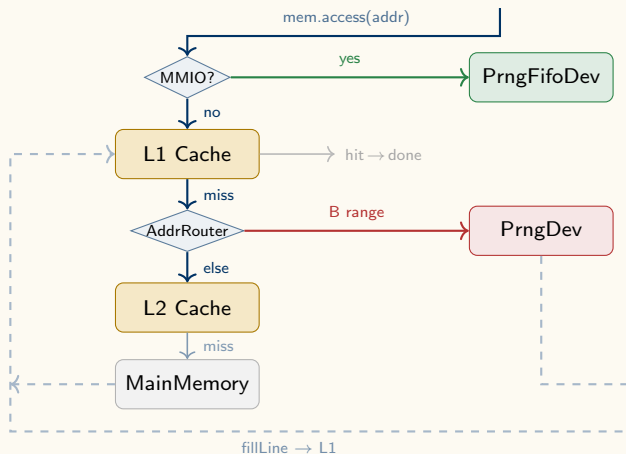
- **From memory** — B cached normally like A and C
- **PRNG FIFO** — hardware stream device generates values on the fly; zero cache footprint ($\rho \rightarrow 0$)

Configurable parameters

- L1 / L2: size, line size, associativity, replacement policy
- Element widths A_p, B_p (bytes per element)
- Tile dimensions T_M, T_N, T_K
- Register tile $\text{REG}_M \times \text{REG}_N \times \text{REG}_K$

Simulator Architecture

MemoryHierarchy



Let's Test the Theory

We have closed-form predictions for L1 traffic:

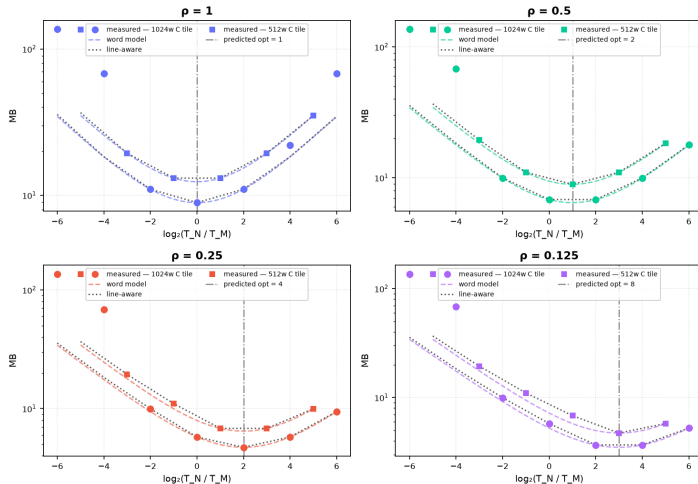
$$\text{reads} = mnk \left(\frac{A_p}{T_N} + \frac{B_p}{T_M} \right) + mn \cdot C_p \quad \text{writes} = mn \cdot C_p$$

with the optimum at $T_N/T_M = 1/\rho$.

We run the simulator over a sweep of tile aspect ratios and overlay the measured traffic against the closed-form curves.

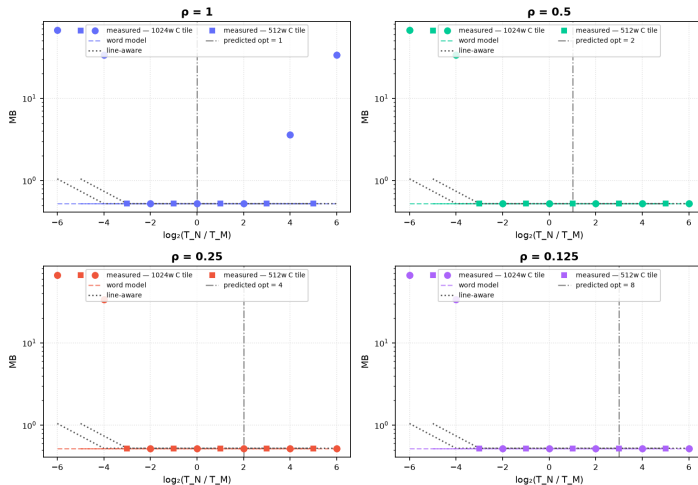
L1 Reads vs. Paper Prediction — Fully-Associative

L1 reads (BytesIn) vs paper prediction — fully-associative cache
 $m=n=k=256$, $T_K=256$, outer-products order



L1 Writes vs. Paper Prediction — Fully-Associative

L1 writes (BytesOut) vs paper prediction — fully-associative cache
 $m=n=k=256$, $T_K=256$, outer-products order



Why A and B Traffic Must Be Equal at the Optimum

Separate the two *input* terms from the reads formula:

$$\underbrace{\frac{mnk A_p}{T_N}}_{\text{A traffic}} + \underbrace{\frac{mnk B_p}{T_M}}_{\text{B traffic}} + mnC_p$$

Their ratio (word model):

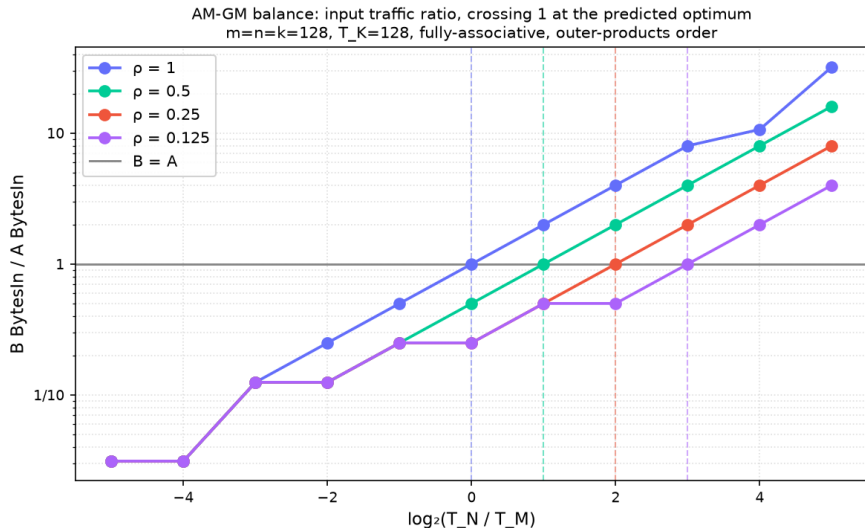
$$\frac{\text{B traffic}}{\text{A traffic}} = \frac{B_p/T_M}{A_p/T_N} = \frac{B_p}{A_p} \cdot \frac{T_N}{T_M} = \rho \cdot \frac{T_N}{T_M}$$

Plug in the optimal ratio $T_N/T_M = 1/\rho$:

$$\left. \frac{\text{B traffic}}{\text{A traffic}} \right|_{\text{opt}} = \rho \cdot \frac{1}{\rho} = \boxed{1}$$

The AM-GM step in the derivation is exactly this balance condition — the minimum is achieved when the two input terms carry *equal* traffic.

The Optimum Balances A and B Traffic Equally



When Does the Paper's Model Hold?

Recall the core assumption of the fast-memory model:

Paper's assumption

The C block occupies essentially **all** of fast memory M . Subrows of A and subcolumns of B stream *through* — they are used once and evicted; they do not compete with C for residency.

This breaks when the C tile is **too large**:

- A strip has T_M rows of k elements — it needs some L1 space too
- If the C tile already fills most of L1, the streaming lines evict C before the block is fully accumulated $\rightarrow$ extra re-fetches $\rightarrow$ model underpredicts traffic

Question: how large can the C tile actually be before the model breaks?

We fix the aspect ratio at the predicted optimum ($T_N/T_M = 4$, $\rho = \frac{1}{4}$) and grow the C tile, measuring excess = measured/predicted. Excess = 1 means the model holds exactly.

Model Validity: Traffic Excess vs. C-Tile Budget

